

Ex.no:2	EDA-DATA IMPORT AND EXPORT

Aim:

To inspect dataset features, detect null values, and export data into CSV, Excel, and SQLite database formats.

Algorithm:

1. Import pandas and sqlite3 libraries.
2. Read diamonds.csv using `pd.read_csv()`.
3. Inspect data structure using `.head()`, `.tail()`, `.info()`, `.describe()`, and `.isnull().sum()`.
4. Export DataFrame to .csv and .xlsx formats using `.to_csv()` and `.to_excel()`.
5. Connect to hotel_booking.db using sqlite3 and write data into an SQL table via `.to_sql()`.
6. Close the database connection and end execution.

Program:

```
import pandas as pd

import zipfile

import io

import os


zip_file_path = '/content/Diamonds Prices2022.csv.zip' # Updated to use the user-provided file
csv_file_name = 'Diamonds Prices2022.csv' # Assuming this is the name of the CSV inside the zip


# Ensure the zip file exists before trying to open it
if not os.path.exists(zip_file_path):

    print(f'Error: The specified file '{zip_file_path}' was not found.')
else:

    # Open the ZIP file
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:

        # Check if the CSV file exists inside the ZIP
        if csv_file_name in zip_ref.namelist():

            # Extract the CSV file content
            with zip_ref.open(csv_file_name) as csv_file:

                # Read the CSV content into a pandas DataFrame
                df = pd.read_csv(csv_file)

                print(f'Successfully extracted and loaded '{csv_file_name}' into a DataFrame.')

                display(df.head())

            else:

                print(f'Error: '{csv_file_name}' not found inside '{zip_file_path}'. Please check the CSV file name within the zip.')


print(df.info())

display(df.describe())

print("\nMissing values per column:")
```

```
display(df.isnull().sum())
```

```
import sqlite3
```

```
# Create a new in-memory SQLite database connection for this specific query
```

```
conn_carat_query = sqlite3.connect(':memory:')
```

```
# Load the original 'df' DataFrame into a new SQL table named 'diamonds_carat_data'
```

```
df.to_sql('diamonds_carat_data', conn_carat_query, index=False, if_exists='replace')
```

```
# Define the new SQL query to get average and max carat by clarity and color
```

```
sql_query_clarity_color_carat = """
```

```
SELECT
```

```
    clarity,
```

```
    color,
```

```
    AVG(carat) AS average_carat,
```

```
    MAX(carat) AS maximum_carat,
```

```
    COUNT(*) AS number_of_diamonds
```

```
FROM
```

```
    diamonds_carat_data
```

```
GROUP BY
```

```
    clarity, color
```

```
ORDER BY
```

```
    average_carat DESC;
```

```
"""
```

```
print("Executing the following SQL query on the 'diamonds_carat_data' table:")
```

```
print(sql_query_clarity_color_carat)
```

```
# Execute the new SQL query
```

```
clarity_color_carat_results = pd.read_sql_query(sql_query_clarity_color_carat, conn_carat_query)
```

```
# Display the results of the new query
```

```
print("\nResults of the new SQL query:")
```

```
display(clarity_color_carat_results)
```

```
# Close the connection
```

```
conn_carat_query.close()
```

```
print("\nSQLite connection closed for this query.")
```

Output:

Successfully extracted and loaded 'Diamonds Prices2022.csv' into a DataFrame.

	Unnamed: 0	carat	cut	color	clarity	depth	table	price	x	y	z
0	1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      53940 non-null  int64
1   carat           53940 non-null  float64
2   cut             53940 non-null  object
3   color           53940 non-null  object
4   clarity         53940 non-null  object
5   depth           53940 non-null  float64
6   table           53940 non-null  float64
7   price           53940 non-null  int64
8   x               53940 non-null  float64
9   y               53940 non-null  float64
10  z               53940 non-null  float64
dtypes: float64(6), int64(2), object(3)
memory usage: 4.5+ MB
None
```

Dropped 'Unnamed: 0' column.

Number of rows with x, y, or z equal to 0: 20
Removed rows where x, y, or z dimensions were 0.

New DataFrame shape: (53923, 10)

DataFrame info after cleaning:
<class 'pandas.core.frame.DataFrame'>
Index: 53923 entries, 0 to 53942
Data columns (total 10 columns):
Column Non-Null Count Dtype
--- ---
0 carat 53923 non-null float64
1 cut 53923 non-null object
2 color 53923 non-null object
3 clarity 53923 non-null object
4 depth 53923 non-null float64
5 table 53923 non-null float64
6 price 53923 non-null int64
7 x 53923 non-null float64
8 y 53923 non-null float64
9 z 53923 non-null float64
dtypes: float64(6), int64(1), object(3)
memory usage: 4.5+ MB
None

Descriptive statistics after cleaning:

	Unnamed: 0	carat	depth	table	price	x	y	z
count	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000
mean	26970.500000	0.797940	61.749405	57.457184	3932.799722	5.731157	5.734526	3.538734
std	15571.281097	0.474011	1.432621	2.234491	3989.439738	1.121761	1.142135	0.705699
min	1.000000	0.200000	43.000000	43.000000	326.000000	0.000000	0.000000	0.000000
25%	13485.750000	0.400000	61.000000	56.000000	950.000000	4.710000	4.720000	2.910000
50%	26970.500000	0.700000	61.800000	57.000000	2401.000000	5.700000	5.710000	3.530000
75%	40455.250000	1.040000	62.500000	59.000000	5324.250000	6.540000	6.540000	4.040000
max	53940.000000	5.010000	79.000000	95.000000	18823.000000	10.740000	58.900000	31.800000

Executing the following SQL query on the 'diamonds_carat_data' table:

```
SELECT
    clarity,
    color,
    AVG(carat) AS average_carat,
    MAX(carat) AS maximum_carat,
    COUNT(*) AS number_of_diamonds
FROM
    diamonds_carat_data
GROUP BY
    clarity, color
ORDER BY
    average_carat DESC;
```

Results of the new SQL query:

	clarity	color	average_carat	maximum_carat	number_of_diamonds
0	I1	J	1.750600	5.01	50
1	I1	H	1.440994	4.13	161
2	I1	I	1.439239	4.01	92
3	SI2	J	1.424259	3.01	479
4	SI2	I	1.391283	3.04	904
5	SI2	H	1.235652	3.01	1550
6	I1	G	1.221419	3.00	148
7	SI1	J	1.172827	2.57	750
8	VS2	J	1.132757	3.51	729
9	I1	D	1.117143	3.40	42
10	I1	E	1.102079	3.05	101
11	I1	F	1.085594	3.01	143
12	SI1	I	1.075099	2.53	1420
13	SI2	G	1.064617	3.01	1540
14	VS2	I	1.062230	2.50	1166
15	VVS2	J	1.033308	2.07	130
16	VS1	J	1.021336	2.59	539
17	SI1	H	0.990009	2.53	2267
18	SI2	F	0.987678	2.67	1598
19	SI2	E	0.922171	2.71	1704

36	VVS2	G	0.655107	1.76	1443
37	VS2	E	0.592744	1.77	2464
38	VVS2	F	0.589619	1.53	972
39	VVS2	H	0.583858	1.86	604
40	VS1	D	0.583021	1.55	705
41	VS1	E	0.573411	1.69	1281
42	VS2	D	0.558941	2.00	1691
43	VVS1	I	0.554930	2.08	355
44	VVS1	G	0.536884	1.70	995
45	VVS2	D	0.528590	1.45	553
46	IF	I	0.515944	1.52	143
47	IF	E	0.506266	1.34	158
48	IF	H	0.506040	1.66	298
49	VVS1	F	0.495327	1.51	734
50	IF	G	0.492345	1.65	678
51	VVS1	H	0.481046	1.70	583
52	VVS1	D	0.477229	1.31	249
53	VVS2	E	0.475935	1.55	989
54	IF	F	0.461567	1.51	383
55	VVS1	E	0.425808	1.26	656

SQLite connection closed for this query.

Executing the following SQL query on the 'diamonds_data' table:

```
SELECT
  cut,
  color,
  AVG(price) AS average_price,
  MAX(price) AS maximum_price,
  COUNT(*) AS number_of_diamonds
FROM
  diamonds_data
GROUP BY
  cut, color
ORDER BY
  average_price DESC;
```

Results of the new SQL query:

	cut	color	average_price	maximum_price	number_of_diamonds
0	Premium	J	6287.733251	18710	806
1	Premium	I	5927.807178	18823	1421
2	Very Good	I	5257.477140	18500	1203
3	Premium	H	5199.486786	18795	2346
4	Very Good	J	5110.304284	18430	677
5	Fair	H	5091.307692	18565	299
6	Good	I	5032.471042	18707	518
7	Fair	J	4975.655462	18531	119
8	Ideal	J	4922.871365	18508	894
9	Fair	I	4607.534483	18242	174
10	Good	J	4587.833333	18325	306
11	Very Good	H	4524.497250	18803	1818
12	Premium	G	4500.249400	18741	2915
13	Ideal	I	4452.125359	18779	2090
14	Premium	F	4327.508621	18791	2320
15	Fair	D	4291.061350	16386	163

Result: